Conductor loops

Reduce delivery latency by owning waiting states

? The biggest waste is often waiting time around coding work

? A deterministic conductor can cut dead time between state changes

? Humans should focus on judgment instead of babysitting workflow transitions

The real problem

Most teams do not lose time because every step is hard.

They lose time because the next step is already obvious, but nobody notices the state change quickly enough.

Examples:

- ? a review landed, but nobody resumed for hours
- ? CI turned green, but the PR stayed idle
- ? a fix was obvious, but the remediation loop stalled
- ? a draft was ready, but the ready-state transition never happened
- ? an approval happened, but the next slice did not start

The hidden tax is waiting

The expensive part is often:

- ? waiting
- ? missing that waiting is over
- ? reloading context after idle gaps
- ? manually pushing the next predictable transition

Each delay looks small on its own.

Across many PRs, reviews, and loops, it turns into:

- ? slower throughput
- ? more interrupted focus
- ? more stale work
- ? longer lead time with no real quality gain

Core idea

Use a conductor-led state machine with deterministic tooling so that waiting states are explicitly owned.

The conductor should:

- ? know the current state
- ? know the next safe transition
- ? resume immediately when the state changes
- ? keep ownership through waits instead of silently dropping it

The real target is the dead time around coding.

The state machine is the core product

The most important idea is not ?we added some AI loops.?

The core idea is simple:

- ? model the workflow explicitly as states and transitions
- ? make the transitions deterministic
- ? keep ownership alive through waiting states
- ? route work back into the right loop automatically

Without the state machine, the workflow depends on:

- ? prompt conventions
- ? memory
- ? manual babysitting
- ? hidden handoff gaps

With the state machine, the workflow becomes:

- ? visible
- ? testable
- ? inspectable
- ? resumable
- ? easier to improve

What humans should focus on

Humans should spend attention where judgment matters most:

- ? architecture
- ? requirements and PRD shaping
- ? acceptance criteria and definition of done
- ? manual testing and exploratory validation
- ? business tradeoffs
- ? final approval and accountability gates

Humans should not spend most of their time on:

- ? polling status
- ? re-requesting predictable transitions
- ? watching CI turn green
- ? noticing routine review-state changes
- ? babysitting loops that already know the next step

What the conductor should own

The conductor should own:

- ? state transitions
- ? waiting-state monitoring
- ? deterministic review choreography
- ? resume / attach / continue decisions
- ? visible state projection back to PRs and trackers
- ? safe stop vs resume decisions after merge

Workers can stay bounded.

The conductor keeps the orchestration truth.

Workflow pattern

- ? intake and refinement
- ? bounded slice planning
- ? local implementation
- ? draft PR
- ? initial local fan-out review
- ? ready-for-review transition
- ? explicit Copilot request and review loop
- ? final DIY DRY/KISS/YAGNI gate
- ? human approval wait
- ? merge
- ? stop or resume next slice

The key is that waiting states are real states, not invisible idle time.

Full walkthrough: before coding starts

The conductor loop should begin before implementation.

It should own:

- ? intake
- ? overlap / duplicate scan
- ? issue refinement
- ? scope clarification
- ? slice shaping
- ? proposal / execution-plan freeze
- ? readiness to start a bounded slice

This matters because a lot of waste starts even before code:

- ? the issue is vague
- ? the slice is too broad
- ? the acceptance criteria are incomplete
- ? the team starts implementation before the work is shaped enough

Full walkthrough: local implementation loop

Once a bounded slice is ready, the conductor moves into the local execution loop:

- ? activate a local worktree / owned slice
- ? implement locally
- ? validate locally
- ? keep ownership while the slice is still local
- ? only open a PR once the slice is integration-ready

The goal is:

- ? local-first work
- ? GitHub only when the slice is ready enough
- ? no half-shaped PR churn

Full walkthrough: draft PR loop

A draft PR should trigger the first real PR-stage loop:

? open PR in draft
? run the initial draft-stage fan-out
? review against:
? SRP / cohesion / boundary quality
? issue scope fit
? AC compliance
? DoD compliance
? architecture fit
? test adequacy
? if needed, route back into the local fix loop
? only move to ready when the draft gate is clean

Full walkthrough: ready-state review loop

Once the PR is marked ready:

- ? the loop must explicitly request or confirm Copilot review
- ? the conductor must enter the Copilot review state
- ? if Copilot comments appear:
- ? run the fix loop
- ? validate
- ? push
- ? re-request
- ? repeat

This matters because:

- ? ready-for-review marks a real workflow transition
- ? the loop should treat it as the entry into the explicit Copilot review gate

Full walkthrough: final approval loop

After Copilot converges:

- ? run the final DIY fan-out
- ? use the final lenses:
- ? DRY
- ? KISS
- ? YAGNI
- ? if clean, enter the human approval wait
- ? if not clean, route back into the local fix loop

Then:

- ? wait for human approval
- ? wait for merge
- ? detect whether merge is terminal or resumable

The loops inside the loop

The conductor is not one single flat flow.

It coordinates multiple nested loops:

- ? refinement loop
- ? slice-shaping loop
- ? local implementation loop
- ? draft-stage review loop
- ? Copilot review/fix loop
- ? final DIY approval loop
- ? merge / closeout / resume loop

The state machine matters because it tells us:

- ? which loop is currently active
- ? what event ends that loop
- ? where the work returns next

Review choreography matters

The loop should distinguish between two different local fan-out gates.

Draft-stage fan-out

Focus on:

- ? SRP / cohesion / boundaries
- ? issue scope fit
- ? AC compliance
- ? DoD compliance
- ? architecture fit
- ? test adequacy

Final pre-approval fan-out

Focus on:

- ? DRY
- ? KISS
- ? YAGNI

These are different gates with different purposes.

Deterministic tooling required

To make this trustworthy, the loop needs deterministic tooling for:

- ? explicit conductor states and transitions
- ? intake / refinement / shaping transitions
- ? draft / ready / Copilot / approval / merge transitions
- ? live conductor plus watcher ownership
- ? visible PR comments on meaningful local state changes
- ? durable local state and closeout artifacts
- ? terminal vs resumable merge decisions
- ? mid-flight steering and safe-point handling
- ? reliable latest-turn / active-question grounding

Without these pieces, the loop looks autonomous but is not actually reliable.

Why this is a company-scale gain

In a company setting, people context-switch constantly.

That means the cost of missed state changes is much higher:

? more PRs in flight
? more reviewers
? more queues
? more partial waits
? more delays between a state change and the next obvious action

A conductor reduces that latency tax.

That can produce:

? shorter cycle time
? less idle PR time
? fewer dropped handoffs
? better developer focus

The human value proposition

Better framing:

? AI owns the predictable coordination gaps

? humans keep judgment, ambiguity, and final accountability

Humans keep:

? judgment

? design

? risk evaluation

? ambiguous decision points

? final accountability

The conductor owns:

? transitions

? waiting-state detection

? routine follow-through

? deterministic next-step orchestration

Pilot evidence already shows the shape

Early pilot runs exposed real orchestration gaps, including:

- ? kickoff that created owned state but did not stay live
- ? watcher-only ownership not being enough
- ? draft / ready / approval states being misclassified
- ? missing mandatory review gates
- ? merge detection without clear stop vs resume semantics
- ? weak post-merge visibility in some slices

This is good evidence.

It means the missing pieces are now concrete and fixable.

Why this is worth doing now

If we solve this well, we do not just save a few minutes.

We reduce one of the most expensive hidden problems in software delivery:

passive delay after a state change

That means:

- ? fewer lost hours
- ? less human polling
- ? faster flow through reviews and fixes
- ? better use of expert attention

That is why this could change how teams actually work day to day.

Practical rollout idea

Start with bounded slices and prove the loop on real work:

- ? one conductor
- ? bounded workers
- ? explicit refinement and review gates
- ? visible PR-side state comments
- ? manual approval retained
- ? deterministic closeout artifacts

Do not aim for magic autonomy first.

Aim for:

- ? trustworthy state ownership
- ? reliable waiting-state handling
- ? faster resume after every state change
- ? a state machine that can be inspected and improved over time

Bottom line

The goal is to eliminate the dead time between implementation steps.

If we do that well:

- ? developers focus on the work only humans should do
- ? the conductor owns the predictable coordination work
- ? delivery becomes faster without lowering quality

The biggest win is cutting latency between state changes.